

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФГАОУ ВО «СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»
ИНСТИТУТ ПЕРСПЕКТИВНОЙ ИНЖЕНЕРИИ
ДЕПАРТАМЕНТ ЦИФРОВЫХ, РОБОТЕХНИЧЕСКИХ
СИСТЕМ И ЭЛЕКТРОНИКИ

КУРСОВАЯ РАБОТА

по дисциплине

«ИНТЕРНЕТ ТЕХНОЛОГИИ»

на тему:

«Разработка веб-приложения для сравнения цен»

Выполнила:

Дмитриева Ксения Николаевна

Студентка 4 курса группы ПИН-б-з-22-1

Направления подготовки

09.03.03 Прикладная информатика

заочной формы обучения

(подпись)

Руководитель работы:

Щеголев А.А.

(ФИО)

Работа допущена к защите _____
(подпись руководителя) (дата)

Работа выполнена и
защищена с оценкой _____ Дата защиты _____

Члены комиссии: _____
(должность) (подпись) (И.О. Фамилия)

Ставрополь, 2026 г

УТВЕРЖДАЮ
И.о. директора департамента цифровых,
робототехнических систем
и электроники
Азаров И.В.

Институт Перспективной инженерии
Департамент цифровых, робототехнических систем и электроники
Направление подготовки 09.03.03 Прикладная информатика
Направленность (профиль) Прикладная информатика в экономике

ЗАДАНИЕ

на курсовую работу

студентки Дмитриевой Ксении Николаевны
(*фамилия, имя, отчество*)

по дисциплине Интернет-технологии

1. Тема работы «Разработка веб-приложения для сравнения цен»

2. Цель- разработать веб-приложение для сравнения цен, обеспечивающее создание, просмотр, изменение статусов и удаление записей о доставках.

3. Задачи Сбор и анализ информации: Поиск и изучение литературы, статей, научных исследований и других источников информации по выбранной теме.
Планирование работы: Составление плана курсовой работы, определение основных этапов и сроков выполнения. Написание текста: Оформление результатов исследования в виде структурированного текста, включающего введение, основную часть и заключение. Проведение экспериментов и практическая реализация: Если это необходимо, выполнение практических заданий, связанных с разработкой программного обеспечения, созданием веб-сайтов или проведением экспериментов. Оформление работы: Соблюдение требований к оформлению курсовой работы, включая правильное оформление списка литературы, таблиц, рисунков и схем. Представление и защита работы: Подготовка презентации и выступление перед комиссией, где студент демонстрирует результаты своего труда и отвечает на вопросы.

4. Перечень подлежащих разработке вопросов:

а) по теоретической части Исторический обзор развития интернет-технологий (в зависимости от выбранной темы: стека технологий). Основные понятия и определения. Описание выбранного стека технологий. Анализ существующих решений и подходов.

б) по практической части Постановка задачи. Выбор методов и инструментов для решения задачи. Описание процесса разработки. Результаты эксперимента или практической реализации. Оценка эффективности предложенных решений.

5. Исходные данные:

а) по литературным источникам _____

б) по вариантам, разработанным преподавателем _____

в) иное _____

6. Список рекомендуемой литературы _____

7. Контрольные сроки представления отдельных разделов курсовой работы:

25 % - _____ “ ____ ” _____ 20_ г.

50 % - _____ “ ____ ” _____ 20_ г.

75 % - _____ “ ____ ” _____ 20_ г.

100 % - _____ “ ____ ” _____ 20_ г.

8. Срок защиты студентом курсовой работы “ ____ ” _____ 20_ г.

Дата выдачи задания “ ____ ” _____ 20_ г.

Руководитель курсовой работы

Старший преподаватель департамента цифровых, робототехнических систем
и электроники института перспективной инженерии Щеголев А.А.

(ученая степень, звание)(личная подпись)(инициалы, фамилия)

Задание принял к исполнению студент заочной формы обучения 4 курса
ПИН-6-3-22-1 группы _____

(личная подпись)

(инициалы, фамилия)

Оглавление	
ВВЕДЕНИЕ.....	5
1 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ.....	8
1.1 Объект и предмет исследования.....	8
1.2 Описание выбранного стека технологий	10
1.3 Анализ существующих решений для сравнения цен	12
2 РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ ДЛЯ СРАВНЕНИЯ ЦЕН	16
2.1 Постановка задачи и системные требования	16
2.2 Обоснование выбора стека технологий	17
2.3 Проектирование архитектуры и базы данных	17
2.4 Разработка серверной части.....	19
2.5 Разработка клиентской части и верстка	24
2.6 Автоматизация процессов разработки и развертывания.....	26
2.7 Тестирование и верификация работоспособности	26
ЗАКЛЮЧЕНИЕ	28
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	30

ВВЕДЕНИЕ

Одной из актуальных задач пользователей является возможность быстрого поиска и выбора наиболее выгодных предложений среди множества интернет-магазинов. Современные веб-приложения для сравнения цен повышают удобство совершения покупок, помогают экономить время и средства, снижают необходимость ручного поиска предложений на различных торговых площадках.

Актуальность темы обусловлена развитием электронной коммерции и увеличением количества интернет-магазинов, что приводит к необходимости поиска эффективных инструментов для анализа и сравнения стоимости товаров. Использование современных веб-технологий, HTML, CSS, Node.js, позволяет создавать веб-приложения для автоматизации процесса сравнения цен.

Объект исследования — процесс сравнения цен посредством веб-технологий.

Предмет исследования — методы и средства разработки веб-приложения для сравнения цен.

Цель работы — разработка и документирование веб-приложения для сравнения цен, демонстрирующего практическое применение технологий Node.js, Express, HTML, CSS, а также использование системы контроля версий Git.

Для достижения цели поставлены следующие задачи:

- проанализировать предметной области и существующие сервисы сравнения цен;
- спроектировать архитектуру приложения и структуру базы данных;
- обосновать выбор инструментальных средств для реализации клиентской и серверной части;
- разработать серверную часть приложения на Node.js с использованием Express, реализующую REST API для сравнения цен;

- разработать клиентскую часть приложения на HTML и CSS с использованием JavaScript для взаимодействия с API;
- использовать систему контроля версии Git и реализовать стратегию ветвления Git Flow для управления версиями;
- провести тестирование функциональности приложения, оценить эффективность.

Методы исследования:

- анализ литературных источников и технической документации;
- изучение существующих веб-сервисов для поиска и сравнения цен;
- сравнительный анализ инструментов разработки;
- проектирование архитектуры веб-приложений, программирование с использованием JavaScript (Node.js и клиентский JavaScript);
- ручное тестирование функциональности разработанного приложения.

Теоретическая значимость работы заключается в систематизации знаний о современных подходах к разработке веб-приложений на основе Node.js, HTML и CSS, а также в изучении принципов построения клиент-серверной архитектуры и организации взаимодействия между пользовательским интерфейсом и серверной частью.

Практическая значимость состоит в создании работоспособного веб-приложения для сравнения цен, которое может быть использовано в качестве основы для дальнейшего развития коммерческого сервиса или учебного проекта при изучении современных технологий веб-разработки. Полученный в ходе разработки опыт может применяться при создании более сложных информационных систем, включающих работу с базами данных, внешними API, авторизацией пользователей и аналитикой изменения цен.

Структура работы соответствует логике исследования и включает введение, две главы (теоретическую и практическую), заключение, список использованных источников и приложения. В первой главе рассматриваются

теоретические основы веб-разработки, основные понятия, используемые при создании веб-приложений, описывается выбранный стек технологий и проводится анализ существующих решений для сравнения цен. Во второй главе подробно рассматриваются этапы практической реализации приложения: постановка задачи, выбор инструментов разработки, проектирование структуры приложения, разработка серверной и клиентской частей, тестирование функциональности и использование системы контроля версий Git. В заключении подводятся итоги выполненной работы, формулируются выводы и определяются возможные направления дальнейшего развития веб-приложения.

1 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

1.1 Объект и предмет исследования

Основные понятия, используемые при изучении объекта.

Веб-приложение — это клиент-серверное программное обеспечение, в котором взаимодействие пользователя с системой осуществляется через веб-браузер посредством протоколов HTTP/HTTPS. В отличие от обычного статического сайта, веб-приложение предоставляет интерактивные возможности: обработку запросов, отображение динамической информации, поиск и сравнение данных.

Клиентская часть (frontend) — часть веб-приложения, которая выполняется в браузере пользователя и отвечает за отображение интерфейса и взаимодействие с пользователем.

Серверная часть (backend) — программная часть приложения, выполняемая на сервере. Отвечает за обработку запросов пользователей, выполнение логики приложения, обработку данных и взаимодействие между клиентской частью и сервером.

HTML (HyperText Markup Language) — язык разметки гипертекста, используемый для создания структуры веб-страниц. С помощью HTML определяются основные элементы интерфейса: заголовки, кнопки, поля поиска, таблицы товаров и другие компоненты.

CSS (Cascading Style Sheets) — язык описания внешнего вида веб-страниц. Используется для настройки дизайна приложения, расположения элементов, размеров, шрифтов и адаптации интерфейса под различные устройства.

Node.js — серверная платформа, позволяющая выполнять JavaScript вне браузера. Используется для создания серверной логики веб-приложений, обработки HTTP-запросов и организации взаимодействия между клиентом и сервером.

API (Application Programming Interface) — программный интерфейс, через который клиентская часть взаимодействует с серверной. В веб-

разработке наиболее распространён REST API, оперирующий ресурсами, идентифицируемыми по URL, и использующий стандартные методы HTTP (GET, POST, PUT, DELETE).

HTTP (HyperText Transfer Protocol) — протокол передачи данных, используемый для обмена информацией между браузером пользователя и сервером. Каждый запрос содержит метод, адрес ресурса, заголовки и дополнительные данные, а ответ сервера включает статус выполнения и возвращаемую информацию.

Система контроля версий — инструмент для отслеживания изменений в исходном коде, позволяющий работать в команде, откатывать изменения, создавать ветки.

Хранение данных — в зависимости от сложности приложения, данные могут храниться в файлах, реляционных базах данных (MySQL, PostgreSQL), NoSQL-базах (MongoDB, Redis) или в оперативной памяти. Хранение в памяти (in-memory) используется для прототипирования и учебных целей, так как данные теряются при перезапуске сервера.

DOM (Document Object Model) — объектное представление HTML-документа, позволяющее JavaScript динамически изменять структуру, содержимое и стили страницы.

1.2 Описание выбранного стека технологий

Для реализации курсовой работы был выбран стек технологий, включающий Node.js, Express.js, HTML5, CSS3, JavaScript, MySQL 8 и Git. Выбор обоснован следующими факторами.

Node.js — это среда выполнения JavaScript на стороне сервера, построенная на движке V8 от Google. Основные преимущества:

- асинхронная неблокирующая модель ввода-вывода, позволяющая эффективно обрабатывать большое количество одновременных запросов;
- использование единого языка программирования как на клиентской, так и на серверной стороне приложения;
- развитая экосистема пакетов через менеджера npm (Node Package Manager);
- высокая производительность и широкое распространение в веб-разработке.

В рамках работы Node.js используется для создания HTTP-сервера, обработки запросов пользователей, взаимодействия с базой данных и реализации бизнес-логики веб-приложения для сравнения цен.

Express.js — минималистичный и гибкий веб-фреймворк для Node.js. Он предоставляет удобные средства для создания серверных приложений и REST API. Основные преимущества:

- простая и понятная организация маршрутов;
- поддержка промежуточного программного обеспечения (middleware);
- удобная обработка HTTP-запросов и ответов;
- возможность быстрой разработки серверной части приложения;
- широкое распространение и подробная документация.

В работе Express.js используется для настройки сервера, организации маршрутов, обработки запросов пользователей и взаимодействия с базой данных MySQL.

HTML5 — современный язык гипертекстовой разметки, предназначенный для создания структуры веб-страниц. Основные преимущества:

- семантическая разметка документов;
- поддержка современных элементов интерфейса;
- совместимость со всеми популярными браузерами;
- удобство создания адаптивных веб-страниц.

В проекте HTML5 используется для формирования структуры страниц, отображения товаров, результатов поиска и таблиц сравнения цен.

CSS3 — язык каскадных таблиц стилей, отвечающий за внешний вид веб-страниц. Основные преимущества:

- разделение структуры документа и его оформления;
- возможность создания адаптивного дизайна;
- поддержка современных технологий Flexbox и Grid;
- улучшение визуального восприятия интерфейса.

В работе CSS3 применяется для оформления пользовательского интерфейса и обеспечения корректного отображения приложения на различных устройствах.

JavaScript (клиентский) — язык программирования, обеспечивающий интерактивность веб-приложения. Основные преимущества:

- динамическое обновление содержимого страницы без перезагрузки;
- выполнение асинхронных запросов к серверу через Fetch API;
- обработка действий пользователя в режиме реального времени;
- широкая поддержка современными браузерами.

В разрабатываемом приложении JavaScript используется для реализации поиска товаров, фильтрации данных, отправки запросов к серверу и отображения результатов сравнения цен.

MySQL 8 — современная реляционная система управления базами данных. Основные преимущества:

- высокая производительность и надежность;
- поддержка стандарта SQL;
- обеспечение целостности данных посредством ограничений и транзакций;
- возможность хранения и обработки больших объемов информации;
- широкое распространение в веб-разработке.

В рамках проекта MySQL 8 используется для хранения информации о товарах, категориях, магазинах и ценах. Использование базы данных позволяет организовать эффективный поиск и обработку данных, необходимых для сравнения цен.

Git — система контроля версий и сервис для хранения репозитория исходного кода. Основные преимущества:

- сохранение истории изменений проекта;
- возможность возврата к предыдущим версиям;
- контроль этапов разработки;
- удобное хранение и публикация исходного кода.

В ходе выполнения курсовой работы Git используется для отслеживания изменений проекта, а GitHub – для хранения репозитория и демонстрации результатов разработки.

Таким образом, выбранный стек технологий Node.js, Express.js, HTML5, CSS3, JavaScript, MySQL 8 и Git обеспечивает создание современного веб-приложения для сравнения цен, обладающего удобным пользовательским интерфейсом, высокой производительностью и возможностью дальнейшего расширения функциональности.

1.3 Анализ существующих решений для сравнения цен

На сегодняшний день существует множество веб-сервисов, позволяющих пользователям сравнивать стоимость товаров в различных

интернет-магазинах. Такие системы помогают быстро находить наиболее выгодные предложения, анализировать характеристики товаров и принимать обоснованные решения при покупке. Рассмотрим наиболее популярные решения, представленные на рынке, выделив их основные преимущества и недостатки.

Яндекс Маркет — один из крупнейших сервисов сравнения цен в России. Платформа предоставляет информацию о товарах из различных интернет-магазинов, позволяет сравнивать цены, характеристики и отзывы покупателей.

Достоинства:

- широкий ассортимент товаров;
- удобная система поиска и фильтрации;
- наличие отзывов и рейтингов продавцов;
- возможность сортировки предложений по различным параметрам.

Недостатки:

- некоторые предложения являются рекламными;
- часть функционала ориентирована на работу внутри экосистемы сервиса.

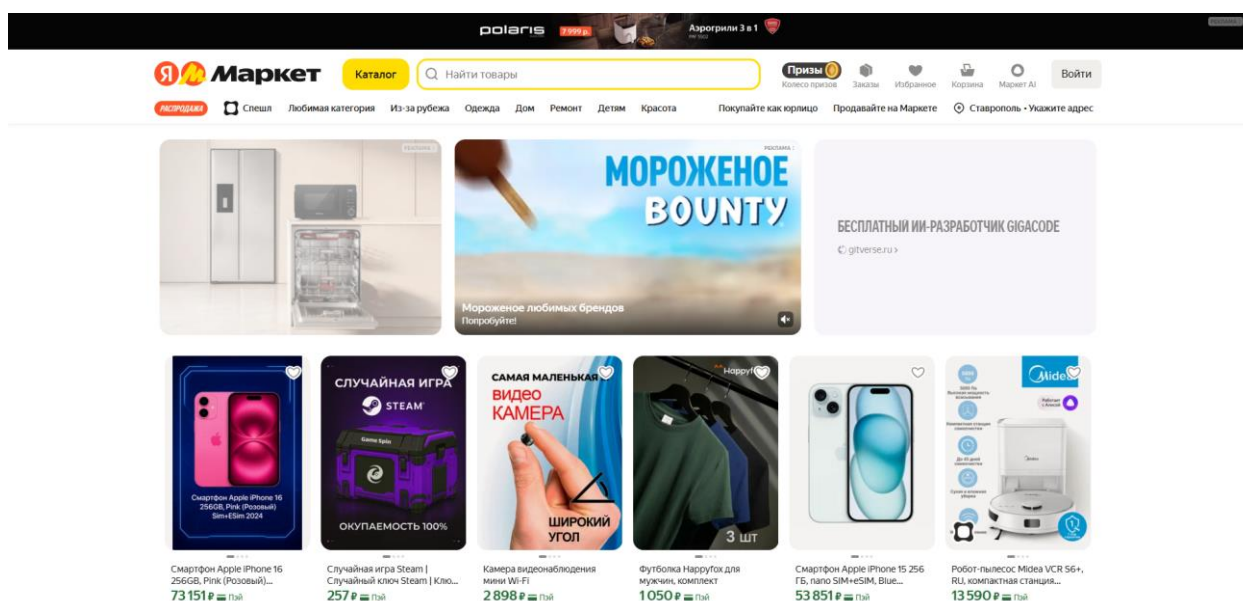


Рисунок 1.1 — Интерфейс сервиса «Яндекс Маркет»

Price.ru — российский сервис сравнения цен, предоставляющий информацию о товарах различных категорий и предложениях множества интернет-магазинов.

Достоинства:

- широкий выбор товаров;
- наличие фильтров и сортировки;
- удобный интерфейс.

Недостатки:

- ограниченные возможности персонализации.

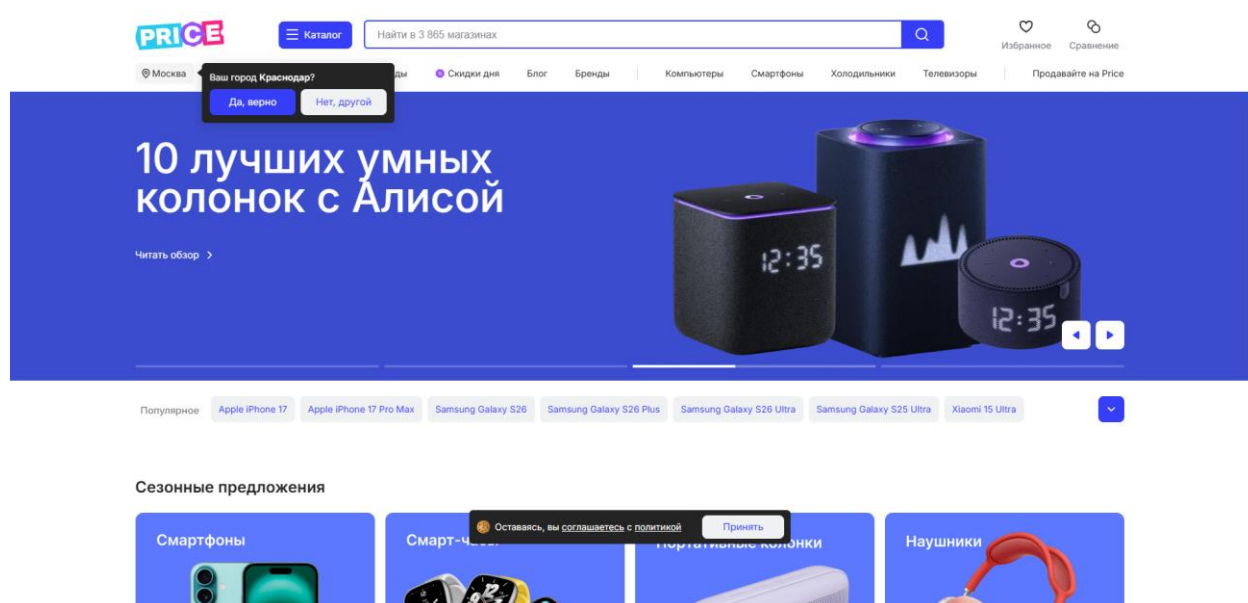


Рисунок 1.2 — Интерфейс сервиса «Price.ru»

E-Katalog — популярный сервис сравнения характеристик и цен на электронику, бытовую технику и другие товары.

Достоинства:

- подробные технические характеристики товаров;
- удобное сравнение моделей;
- большое количество фильтров.

Недостатки:

- основной акцент сделан на технических товарах.

Основная цель — демонстрация принципов построения клиент-серверной архитектуры, работы с базой данных MySQL и взаимодействия между клиентской и серверной частями приложения. Основными преимуществами разрабатываемого приложения является:

- простая и понятная структура;
- возможность самостоятельного изменения функциональности;
- возможность дальнейшего расширения функционала за счет добавления авторизации пользователей;
- возможность дальнейшего расширения функционала за счет добавления истории изменения цен и сравнения;
- системы избранного и интеграции с внешними API интернет-магазинов.

Таким образом, анализ существующих решений показывает, что современные сервисы сравнения цен обладают широкими возможностями, однако часто являются закрытыми коммерческими продуктами. Разработка собственного веб-приложения позволяет получить практические навыки создания информационных систем и создать основу для дальнейшего развития полноценного сервиса сравнения цен.

2 РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ ДЛЯ СРАВНЕНИЯ ЦЕН

2.1 Постановка задачи и системные требования

На основе анализа требований к веб-приложению для сравнения цен была сформулированы следующие задачи: разработать клиент-серверное приложение, позволяющее пользователю выполнять основные операции для сравнения цен, а именно:

- просматривать список всех товаров – отображение наименования товара и цены продажи в рублях;
- создавать товар – ввод наименования товара и цены продажи, отправка данных на сервер;
- просмотр данных выбранного товара – наименование и цену продажи;
- редактировать товар – загрузка текущих данных в модальное окно, сохранению изменений, отправка на сервер;
- удаление товара – удаление после подтверждения пользователя;
- сравнение цены – возможность сравнение цены для всех товаров, просмотр результата о сравнении.

Приложение должно быть реализовано в виде RESTful веб-сервиса, в котором клиентская часть (веб-страница) взаимодействует с серверным API посредством асинхронных HTTP-запросов. Серверная часть обрабатывает запросы пользователей, выполняет операции по получению и сохранению данных в базе данных MySQL 8 и возвращает результаты в формате JSON. Полученные данные динамически отображаются на веб-странице без её полной перезагрузки.

Дополнительные требования:

- использование системы контроля версий Git с регулярными коммитами, отображающими этапы разработки;
- структурированный и читаемый код с комментариями.

2.2 Обоснование выбора стека технологий

Стек HTML, CSS, JavaScript (frontend), Node.js, Express (backend) — один из самых распространённых в веб-разработке, особенно для небольших и средних приложений, прототипов и учебных проектов.

HTML, CSS, JS — фронтенд интерфейс, базовые технологии веб-разработки, используются везде.

Node.js — среда выполнения JavaScript на сервере. Быстрый старт, единый язык JS на клиенте и сервере.

Express — самый популярный фреймворк для Node.js, создание REST API и веб-серверов.

Для хранения данных используется MySQL 8 — реляционная база данных, обеспечивающая целостность данных, быстрые запросы и безопасность. Связка с Node.js и Express обеспечивают современную, легко масштабируемую и поддерживаемую архитектуру приложения.

Система контроля версий Git — самая популярная система контроля версии в индустрии разработки программного обеспечения. Она фактически стала стандартом для большинства IT-команд.

GitHub — самый популярный веб-сервис хостинга IT-проектов и их совместной разработки. Он построен на системе контроля версии Git и предоставляет дополнительные инструменты для управления кодом, совместной работы и автоматизации процессов разработки.

2.3 Проектирование архитектуры и базы данных

В проекте используется MySQL 8 — реляционная база данных.

Для создания базы данных проекта в консоли MySQL Workbench 8.0 CE было прописано команды.

Листинг 1.1 — Создание базы данных

```
CREATE DATABASE compare_price;
USE compare_price;
CREATE TABLE products (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    price DECIMAL(10,2) NOT NULL
);
```

Таблица 1.1 — Структура сущности product

Поле	Тип данных	Ограничения	Описание
id	INT	AUTO_INCREMENT PRIMARY KEY	Уникальный идентификатор товара
name	VARCHAR(255)	NOT NULL	Название товара
price	DECIMAL(10,2)	NOT NULL	Цена товара

Листинг 1.2 — Посмотреть все товары в базе данных

```
SELECT * FROM products;
```

Листинг 1.3 — Добавить товар в базу данных

```
INSERT INTO products (
    name,
    price
)
VALUES (
    'Тестовый товар',
    99.99
);
```

Листинг 1.4 — Удалить товар из базы данных

```
DELETE FROM products
WHERE id = 1;
```

Листинг 1.5 — Обновить товар в базе данных по идентификатору

```
UPDATE products
SET
    name = 'Новый товар',
    price = 150.00
WHERE id = 1;
```

2.4 Разработка серверной части

Скрипты и серверная часть расположены в файлах `script.js`, `result.js`, `products.js`, `database.js`, `server.js`.

Файл `script.js` — является основным клиентским скриптом разрабатываемого веб-приложения. Он реализует всю логику взаимодействия пользователя с интерфейсом страниц `price.html` и `compare.html`, включая все CURD операции и процесс сравнения цен. Скрипт работает с серверным API через `fetch` используя `Express` на бэкенде.

В начале скрипта выполняется подключение DOM-элементов.

Вспомогательные функции включают:

- `randomGradient()` — возвращает случайный CSS-градиент из заранее заданного набора. Используется для фона каждой карточки товара, создавая визуальное разнообразие;
- `clearInputs()` — очищает поля ввода в модальке добавления товара (наименование и цена). Вызывается после успешного добавления товара, чтобы подготовить форму к следующему вводу.

Функции обработчики модальных окон:

- `openModal()` — при клике на панель «Товары +» открывает модальное окно добавления товара, меняя его `display` на `flex`;
- `closeModal()` — закрывает модальку добавления при клике на крестик;
- `closeEditModal()` — закрывает модальку редактирования аналогичным образом.

Все обработчики обернуты в проверку `if (элемент)`, что предотвращает ошибки на страницах, где эти элементы отсутствуют (например, на `compare.html` нет модальки добавления).

Основные CRUD-функции:

- `loadProducts()` — загрузка товаров на страницу прайса. Отправляет GET-запрос к `/products`, получает массив товаров в формате JSON, очищает

контейнер `productsContainer`, для каждого товара создаёт карточку с названием, ценой, кнопками «Изменить» и «Удалить».

Обработчики кнопок:

- «Изменить» — открывает модальку редактирования, подставляя в поля название и цену текущего товара, запоминая его `id` в переменную `currentProductId`.

- «Удалить» — после подтверждения пользователем отправляет DELETE-запрос на сервер и обновляет список (`loadProducts()`).

В случае ошибки (сеть, сервер, парсинг) выводится сообщение в консоль и на страницу.

`loadCompareProducts()` — загрузка товаров на страницу сравнения:

- работает с контейнером `compareProductsContainer`;
- создаёт карточки без кнопок;
- если товаров нет, выводит сообщение «Товары не найдены».

`addProduct.onclick()` — добавление товара, при клике на кнопку «Добавить товар» в модальном окне:

- отправляет POST-запрос на `products` с данными из полей `productName` и `productPrice`.

- закрывает модальное окно;
- очищает поля модального окна;
- обновляет список товаров через `loadProducts()`.

`saveEdit.onclick()` — сохранение изменений, при клике на кнопку «Сохранить» в модальке редактирования:

- отправка PUT-запроса с новыми данными;
- закрытие модального окна;
- обновление списка товаров;
- инициализация при загрузке страницы.

Обработчики кнопок:

- «Сравнение» — при клике на кнопку с классом `comparison-button` (на странице `compare.html`), загружаются все товары с сервера (`GET /products`),

массив копируется и сортируется по возрастанию цены (с помощью `parseFloat` для корректного числового сравнения), отсортированный список сохраняется в `localStorage` под ключом `compareResults`, происходит переход на страницу `result-compare.html`, где данные будут извлечены и отображены.

Этот механизм позволяет сохранить результаты сравнения между страницами без повторного запроса к серверу.

Взаимосвязь с бэкендом.

Все взаимодействия с сервером построены на HTTP-методах RESTful API:

- GET /products — получение списка.
- POST /products — создание нового товара.
- PUT /products/:id — обновление товара.
- DELETE /products/:id — удаление товара.

Сервер (Express + MySQL) обрабатывает эти запросы и возвращает JSON-ответы, которые скрипт обрабатывает дальше.

Файл `result.js` отвечает за отображение результатов сравнения цен на странице `result-compare.html`. Он обрабатывает данные, сохранённые в `localStorage` на предыдущем шаге (после нажатия кнопки «Сравнение» на странице `compare.html`), и генерирует визуальный список карточек товаров, отсортированных по цене, с выделением минимального (MIN) и максимального (MAX) значения.

Загрузка данных из `localStorage` — после полной загрузки DOM скрипт извлекает строку JSON по ключу `compareResults`.

Валидация данных — проверяется наличие данных, их корректность (непустой массив).

Генерация HTML-разметки — для каждого товара создаётся карточка с названием, ценой и меткой (MIN/MAX при необходимости).

Вставка в контейнер — готовый HTML вставляется в элемент `#resultContainer`.

Скрипт обернут в обработчик DOMContentLoaded, который гарантирует, что все элементы DOM полностью загружены до начала выполнения. Это необходимо для безопасного обращения к контейнеру #resultContainer.

Получение данных из localStorage. Извлекается строка JSON, сохранённая ранее на странице compare.html при клике на кнопку «Сравнение». В ней хранится массив товаров, уже отсортированный по возрастанию цены.

Проверка наличия данных. Если данных нет (!data), в контейнер выводится сообщение с предложением вернуться на страницу сравнения. Скрипт завершает выполнение (return).

Если данные есть, они парсятся в массив объектов с помощью JSON.parse(). При ошибке парсинга (невалидный JSON) управление переходит в блок catch, где выводится общее сообщение об ошибке.

Проверка на пустой массив. Если массив products пуст, выводится соответствующее сообщение, и скрипт завершается.

Функция escape(text) — эта функция создаёт временный DOM-элемент, присваивает ему переданный текст через textContent (безопасно) и возвращает innerHTML — экранированную строку, готовую к вставке в разметку. Это предотвращает внедрение вредоносного кода, если название или цена товара содержат HTML-теги.

Генерация HTML-разметки — собирается строка html, которая состоит из:

- внешнего контейнера. compare-scroll-wrapper с горизонтальной прокруткой;
- внутреннего flex-контейнера. compare-items;
- блока карточек для каждого товара.

Для каждого товара (по индексу index) определяются:

- bgClass – CSS-класс фона карточки;
- compare-item-min – для минимальной цены (зелёный фон);

- `compare-item-max` – для максимальной цены (красный фон);
 - `compare-item-default` – для всех остальных (серый фон);
 - `label` – HTML-метка MIN или MAX (зелёная или красная плашка)
- выводится только для первого и последнего элементов (или единственного).

Условия определения:

- если товар один – он получает метку MIN;
- если товар первый в списке (самая низкая цена) – метка MIN;
- если товар последний (самая высокая цена) – метка MAX;
- остальные товары получают нейтральный фон и не имеют метки.

Название и цена безопасно вставляются с помощью `safeName` и `safePrice`, полученных через функцию `escape`. Это гарантирует, что даже если в названии товара есть специальные символы (`<`, `>`, `&` и т.п.), они будут отображаться корректно как текст, а не как HTML-теги.

Собранный HTML присваивается свойству `innerHTML` контейнера `#resultContainer`. Таким образом, все карточки появляются на странице.

Блок `try/catch` перехватывает ошибки парсинга JSON (`JSON.parse`).

В случае любой ошибки в контейнер выводится сообщение «Ошибка загрузки данных.».

В сообщениях используются классы. `error-message`, которые стилизованы в `result-compare.css` для единообразного отображения.

Файл `Products.js` — является роутером Express для обработки HTTP-запросов, связанных с товарами в интернет-магазине. Он определяет RESTful API для выполнения операций CRUD (Create, Read, Update, Delete) над товарами в базе данных MySQL. Роутер подключается к основному серверу (в `server.js`) по пути `/products`, поэтому все маршруты, определённые здесь, доступны относительно этого префикса.

Листинг 1.6 — Метод GET файла products.js

```
// Получить все товары
router.get("/", (req, res) => {
  db.query("SELECT * FROM products", (error, result) => {
    if (error) {
      res.status(500).json(error);
    } else {
      res.json(result);
    }
  });
});

// Получить товар по id
router.get("/:id", (req, res) => {
  const id = req.params.id;
  db.query("SELECT * FROM products WHERE id = ?", [id], (error, result) => {
    if (error) {
      res.status(500).json(error);
      return;
    }
    if (result.length === 0) {
      res.status(404).json({ message: "Товар не найден" });
      return;
    }
    res.json(result[0]);
  });
});
```

Файл Database.js — отвечает за подключение к базе данных MySQL и предоставление объекта подключения (db) для выполнения запросов к БД из других модулей (например, из роутеров products.js). В нём настраивается соединение с сервером MySQL, проверяется его работоспособность и экспортируется готовый экземпляр.

Файл server.js — является главным (входным) файлом серверной части веб-приложения. Он создаёт и настраивает HTTP-сервер на базе Express, подключает маршрутизацию, обслуживание статических файлов и API для работы с товарами. Файл служит "точкой входа" для всего бэкенда — при его запуске сервер начинает слушать входящие запросы и перенаправлять их в соответствующие обработчики.

2.5 Разработка клиентской части и верстка

Клиентская часть расположена в папке pages.

Файл main.html — это главная страница веб-приложения. Она служит точкой входа и содержит только базовую структуру, шапку, навигационное

меню и подвал. Представляет собой статический «визитный листок» с навигацией.

Файл `price.html` — это основная рабочая страница проекта. Она предназначена для управления товарами, реализует полный CRUD-цикл (Create, Read, Update, Delete), создание, просмотр, редактирование и удаления карточки товара. Нажатие на кнопку «Товары +» открывает модальное окно для создания товара, имеющее два поля для ввода, поле с наименованием товара и ценой, и двумя кнопками, кнопкой сохранения и закрытием. Это центральный элемент приложения, через который осуществляется все взаимодействие с базой данных. Основная логика вынесена в `script.js`.

Файл `compare.html` — эта страница сравнения цен проекта. Она предназначена для отображения всех товаров из базы данных в виде карточке без CURD операций, и запуска процесса сравнения по цене. Нажатие на кнопку «Сравнение» запускает процесс сортировки по цене, сохранения отсортированных данных и выполнение перехода на страницу `result-compare.html` с результатом сравнения. Страница служит мостом между просмотром товаров и их сравнением. Основная логика вынесена в `script.js`, что соответствует принципу разделения ответственности.

Файл `result-compare.html` — это страница результата сравнения цен. Она открывается после нажатия кнопки «Сравнение» на странице `compare.html`. На этой странице отображаются товары, отсортированные по цене (от минимальной к максимальной), с визуальным выделением самого дешёвого (MIN) и самого дорогого (MAX) товара. Данные передаются через `localStorage` и обрабатываются скриптом `result.js`.

Все используемые стили хранятся в папке `style`. Папка `style` содержит `base.css`, `price.css`, `compare.css`, `result-compare.css`.

Файл `base.css` — содержит основные стили. Общие сбросы, стиль страницы, шапки, заголовка, меню, шрифтов, футера. К нему подключены `main.html`, `price.html`, `compare.html`, `result-compare.html`.

Файл `price.css` — содержит стили для панели товары, шрифты для панели, сетку карточек товаров, модального окна товара, кнопок редактирования и удаления товара, стили для сообщений ошибок или отсутствия товара. К нему подключены `price.html`, `compare.html`.

Файл `compare.css` — содержит стили для кнопки сравнения, сетку карточек товаров, стили для сообщений ошибок или отсутствия товара. К нему подключены `compare.html`.

Файл `result-compare.css` — содержит стили для страницы, карточек товаров, прокрутке карточек, стиль для карточки с минимальной ценой, стиль для карточки с максимальной ценой, стили для карточек с ценой между минимальной и максимальной. К нему подключены `result-compare.html`.

2.6 Автоматизация процессов разработки и развертывания

Разработка велась с использованием система контроля версий Git. Репозиторий опубликован на онлайн-платформе GitHub под названием WebComparePrice. Производились регулярные коммиты с описанием этапом разработки. Работа велась в ветке `develop`, после завершение работы над проектом было выполнено слияние с веткой `main`. Все коммиты были выгружены в удаленный репозиторий. Репозиторий сделан публичны.

Использование Git позволяет продемонстрировать историю разработки и при необходимости вернуться к предыдущей версии.

2.7 Тестирование и верификация работоспособности

Тестирование проводилось вручную с использованием браузере Google Chrome и инструментов разработчика (вкладка Network для анализа запросов, Console для вывода ошибок). Проверялись следующие сценарии:

- открытие страницы: при запуске, появлялась страница `main.html`;
- переход по страницам через меню: переход по страницам расположенными в меню;

- загрузка товаров: при наличии данных о товарах в базе данных, они должны отображаться на странице price.html, compare.html;
- открытие модального окна: нажатие «Товары +» на странице price.html;
- закрытие модального окна: нажатие креста в открытом модальном окне;
- создание товара: ввод наименования и цены, нажатие «Добавить товар».
- редактирование товара: нажатие «Редактировать», возможность отредактировать данные сохраненного товара и обновить их в базе данных, отобразить обновленные данные на странице price.html, compare.html;
- удаление товара: нажатие «Удалить» и подтверждение в диалоговом окне;
- сравнение товара: нажатие кнопки «Сравнение» на странице compare.html, открытие страницы result-compare с результатом сравнения товаров;

Обработка ошибок:

- попытка открыть удаленный товар: сервер возвращает 404.

Все сценарии были успешно выполнены.

ЗАКЛЮЧЕНИЕ

В ходе работы над проектом было разработано полноценное веб-приложение для сравнения цен на товары. Проект представляет собой законченное решение, включающее клиентскую часть, серверную логику и базу данных.

Спроектирована архитектура приложения — фронтенд на HTML, CSS и JavaScript взаимодействует с сервером через REST API, построенном на Node.js и Express; данные хранятся в MySQL 8.

Реализован полный CRUD товаров — добавление, просмотр, редактирование и удаление записей через удобный интерфейс с модальными окнами.

Организовано сравнение цен пользователь нажатием одной кнопки получает список товаров, отсортированных по возрастанию цены, с визуальным выделением самого дешёвого (MIN) и самого дорогого (MAX) товара.

Создан адаптивный и единообразный интерфейс — стилизованы шапка, меню, карточки товаров, модальные окна, страницы прайса, сравнения и результатов.

Настроена серверная инфраструктура— реализован сервер на Express с раздачей статики, обработчиками страниц и API-маршрутами, настроено подключение к MySQL.

Используемый стек:

- frontend: HTML5, CSS3, JavaScript;
- backend: Node.js, Express;
- база данных: MySQL 8.

Приложение полностью работоспособно и выполняет поставленную задачу. Пользователь может:

- просматривать товары в виде цветных карточек на странице прайса;
- добавлять новые товары, редактировать и удалять существующие;

- перейти на страницу сравнения, увидеть все товары и запустить сравнение цен;

- получить наглядный результат с выделением минимальной и максимальной цены и горизонтальной прокруткой при большом количестве товаров.

Несмотря на завершённость проекта, остаётся ряд перспектив для развития:

- внедрение аутентификации и авторизации;
- добавление категорий и тегов;
- реализация истории сравнений;
- улучшение мобильной адаптивности;
- оптимизация бэкенда;
- добавление пагинации или бесконечной прокрутки при большом количестве товаров.

Проект демонстрирует успешное применение современных веб-технологий для создания функционального инструмента сравнения цен. Он служит хорошим примером архитектуры клиент-серверного приложения с использованием Node.js, Express и MySQL. Благодаря модульной структуре и чистому коду, проект легко расширять и адаптировать под новые требования. Разработанное приложение готово к использованию и может стать основой для более масштабных систем.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1) ГОСТ 2.105-95. Единая система конструкторской документации. Общие требования к текстовым документам. — Введ. 01.07.1996. — М. : Издательство стандартов, 1996. — 35 с.
- 2) ГОСТ 7.32-2017. Система стандартов по информации, библиотечному и издательскому делу. Отчет о научно-исследовательской работе. Структура и правила оформления. — Введ. 01.07.2018. — М. : Стандартинформ, 2017. — 28 с.
- 3) ГОСТ 19.101-77. Единая система программной документации. Виды программ и программных документов. — Введ. 01.01.1980. — М. : Издательство стандартов, 1977. — 12 с.
- 4) ГОСТ 34.201-2020. Информационные технологии. Комплекс стандартов на автоматизированные системы. Виды, комплектность и обозначение документов при создании автоматизированных систем. — Введ. 01.01.2022. — М. : Стандартинформ, 2021. — 24 с.
- 5) ГОСТ Р 7.0.5-2008. Система стандартов по информации, библиотечному и издательскому делу. Библиографическая ссылка. Общие требования и правила составления. — Введ. 01.01.2009. — М. : Стандартинформ, 2008. — 19 с.
- 6) ГОСТ Р 7.0.100-2018. Библиографическая запись. Библиографическое описание. Общие требования и правила составления. — Введ. 01.07.2019. — М. : Стандартинформ, 2018. — 48 с.
- 7) ГОСТ Р 52872-2019. Интернет-ресурсы и другая информация, представленная в электронно-цифровой форме. Приложения для стационарных и мобильных устройств, иные пользовательские интерфейсы. Требования доступности для людей с инвалидностью и других лиц с ограничениями жизнедеятельности. — Введ. 01.01.2020. — М. : Стандартинформ, 2019. — 32 с.
- 8) ГОСТ Р 53622-2009. Информационные технологии. Информационно-вычислительные системы. Стадии и этапы жизненного

цикла, виды и комплектность документов. — Введ. 01.01.2010. — М. : Стандартиформ, 2009. — 16 с.

9) ГОСТ Р 56545-2015. Защита информации. Уязвимости информационных систем. Правила описания уязвимостей. — Введ. 01.12.2015. — М. : Стандартиформ, 2015. — 24 с.

10) СанПиН 2.2.2/2.4.1340-03. Гигиенические требования к персональным электронно-вычислительным машинам и организации работы. — Введ. 30.06.2003. — М. : Минздрав России, 2003. — 36 с.

11) Клэбёрн, Д. Rust. Быстрая разработка приложений / Д. Клэбёрн. — СПб. : Питер, 2021. — 448 с. — ISBN 978-5-4461-1841-2.

12) Бланд, Д. Программирование на Rust: производительность и безопасность / Д. Бланд, Дж. Орендорф, Л. Ф. С. Тиндалл. — 2-е изд. — М. : ДМК Пресс, 2023. — 624 с. — ISBN 978-5-97060-775-4.

13) Макнамара, Т. Rust в действии / Т. Макнамара. — СПб. : БХВ-Петербург, 2022. — 400 с. — ISBN 978-5-9775-1166-7.

14) Моултон, Н. Docker. Глубокое погружение / Н. Моултон. — М. : Питер, 2021. — 350 с. — ISBN 978-5-4461-1523-7.

15) Кочер, П. С. Микросервисы и контейнеры Docker / П. С. Кочер. — М. : ДМК Пресс, 2020. — 320 с. — ISBN 978-5-97060-739-8.

16) Аллен, Г. Полное руководство по SQLite / Г. Аллен, М. Оуэнс. — 2-е изд. — М. : Apress, 2010. — 400 с. — ISBN 978-1-4302-3225-4.

17) Rocket Web Framework Documentation [Электронный ресурс]. — Режим доступа: <https://rocket.rs/>

18) sqlx — Rust async SQL toolkit. Документация [Электронный ресурс]. — Режим доступа: <https://github.com/launchbadge/sqlx>

19) Филдинг, Р. Т. Архитектурные стили и проектирование сетевых программных архитектур / Р. Т. Филдинг. — Калифорнийский университет, Ирвайн, 2000. — 180 с.

20) Репозиторий проекта «Веб-приложение для трекинга доставок» [Электронный ресурс]. — Режим доступа: <https://github.com/chuxan4ik/track-app>